The Unseen Journey Quest

"Trapped in a dark maze, you can only see the ground beneath your feet. Somewhere in the shadows, a hidden key unlocks the way out, but you can't see it, and you don't know where it is. You can sense when you're moving closer to the key, guiding your steps through the darkness."

Objective

In this assignment, you are tasked with creating a 2D maze game. The player is blind and can only sense their surroundings within the maze. The game's grid is represented using a multidimensional linked list. The player must navigate through the maze, find a hidden key, and unlock the exit door, both of which are randomly placed. However, the player cannot see the key or the door; instead, they can sense whether they are getting closer or further from the key using a sensing ability. The game will calculate the number of allowed moves based on the Manhattan Distance (City Block Distance) between the player, the key, and the door. Movement will be restricted by an "undo" feature that allows limited backtracking. A player can gather coins, which are visible, and the difficulty will scale across three levels.

"Can you find the key, unlock the exit, and escape the maze before you're lost in the dark forever?"

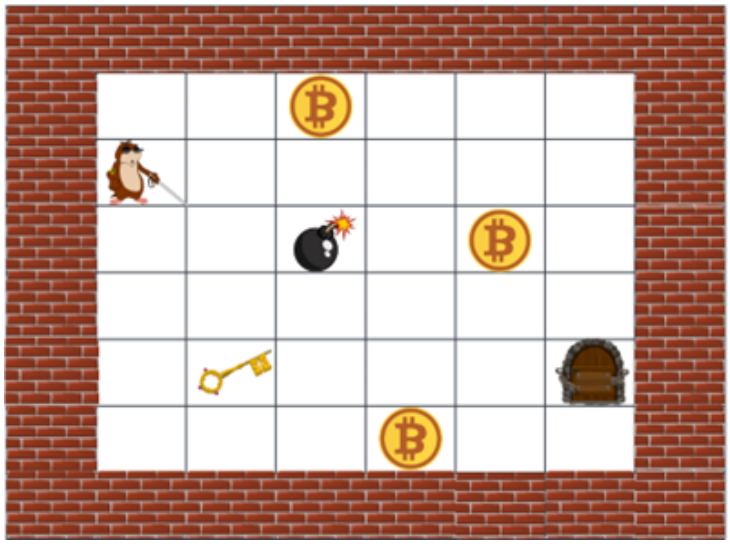

Figure 1:

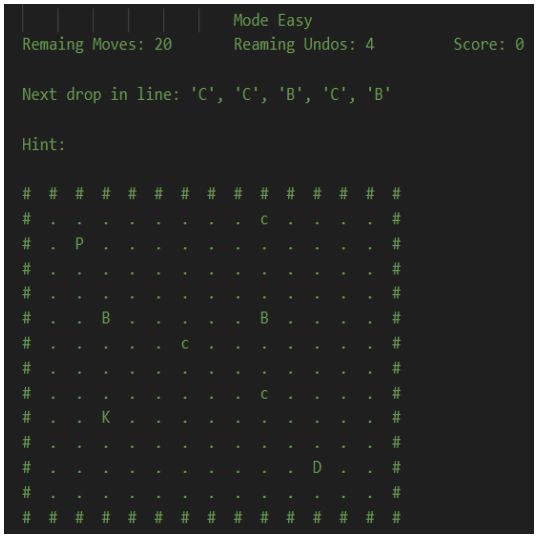

Figure 2: How it will look on console

Game Description

1. Game Board

- The game grid is represented using a multidimensional linked list, where each node represents a grid cell, and each cell is connected to its adjacent cells (up, down, left, right).
- Player, key, door, coins, and bomb will be placed randomly each time the game starts; the code should be generic.
- The player can see the maze's boundaries, but the key and the exit door are invisible.

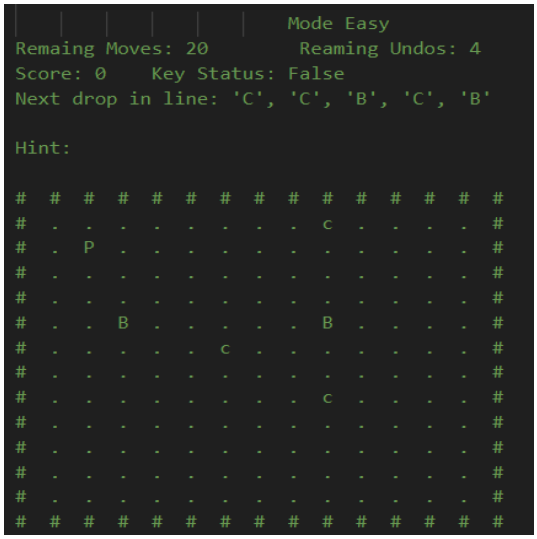

Figure 3:

- The player's current position is visible, and they can move within the boundaries of the grid.

2. Movement

- The player can move in four directions: up, down, left, and right.
- The player can only sense the distance to the hidden key, which is calculated using the City Block Distance formula. This sensing ability informs the player whether they are getting "closer" or "further away" from the key after each move.

  If the player moves closer to the key from their initial position, a prompt will display "Getting Closer." Conversely, if they move further away, the prompt will display "Further away."

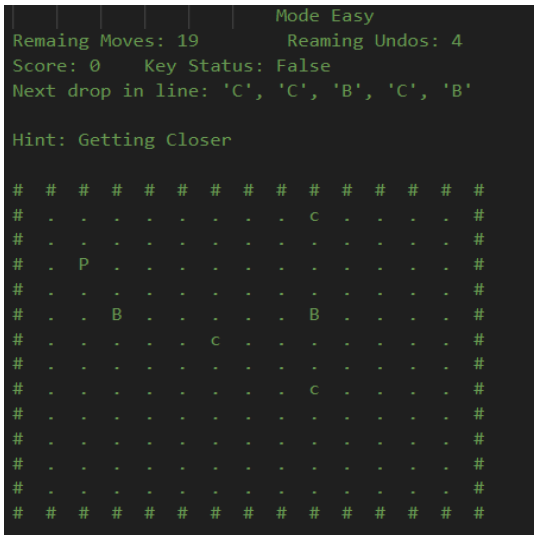

Figure 4: Getting Closer to the Key

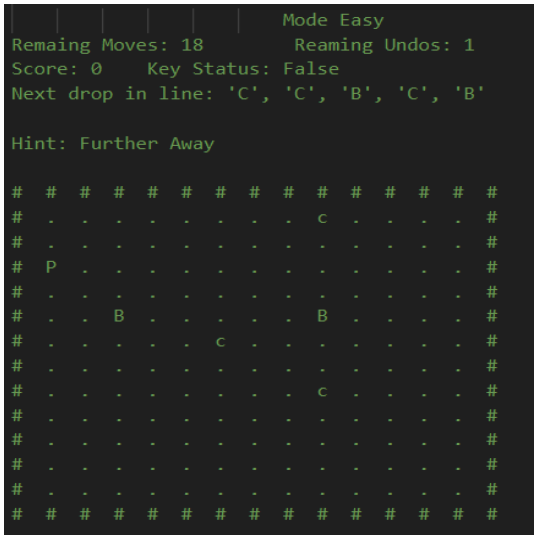

Figure 5: Further Away from the Key

- Once the player moves in one direction, they cannot immediately move back without using an undo feature. The number of moves allowed is based on the City Block Distance between the player, the key, and the door.

- The player has a limited number of moves to find the key and the exit. At the start of the game, the total number of moves will be calculated, including the distance from the player to the key and from the key to the door.

3. Sense Power (Hint System)

- The player has a power that allows them to sense the key's proximity.
- After each move, the game will calculate the City Block Distance between the player's current node and the hidden key. The player will receive a hint indicating whether they are getting "closer" or "further away" from the key.
- The same applies to the door; the player can only enter the door if they have obtained the key.

4. Levels of Difficulty

- **Easy Level:** The player has a calculated number of moves (based on the City Block Distance) plus 6 extra moves. They can use the undo feature 6 times, and the grid size is 10x10.
- **Medium Level:** The player has the calculated number of moves plus 2 extra moves, can use the undo feature 4 times, and the grid size is 15x15.
- **Hard Level:** The player has exactly the calculated number of moves, only 1 undo move, and the grid size is 20x20.

5. Undo Feature

- The player has limited undo powers. Each time the player undoes a move, they return to their previous position by popping their last move from the stack.
- The number of undo actions is limited and depends on the difficulty level.

6. Inventory System

- Coins will appear randomly on the grid. After a short time, they will disappear, and new coins will appear in different locations.
- The player collects coins by moving onto their positions. Each coin collected grants one additional undo move.
- Once the player reaches the exit door, the collected coins will be revealed in the order they were picked up. The reveal will include the exact positions where the coins were collected, showing the order and coordinates (e.g., "(2,3)") of each collected coin, starting with the first coin and ending with the last.

7. Score

- If the player finishes the game with unused moves, each remaining move will contribute 1 point to the player's score.
- Each coin collected will contribute 2 points to the player's score and grant 1 additional undo movement.
- The total score, consisting of remaining moves and collected coins, will contribute to the player's bonus marking in the final evaluation.

8. Game Over Conditions

- If the player steps on a bomb, the game will immediately end, and the player will lose.
- If the player uses up all their allowed moves without reaching the exit door, the game will be over, and the player will lose.
- If the player successfully finds the hidden key and navigates to the exit door within the allowed number of moves, the game will be over, and the player will win.

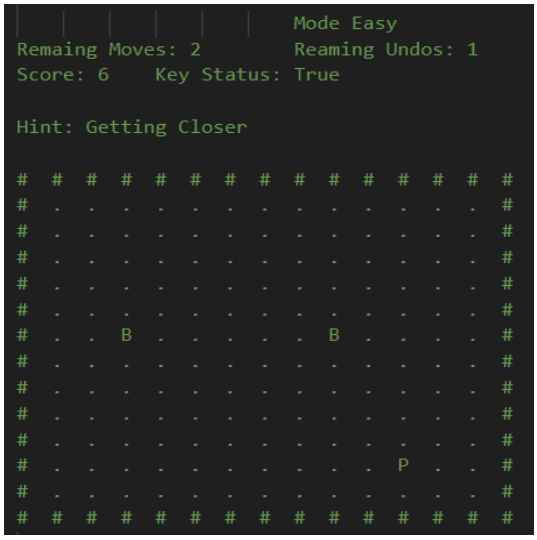

Figure 6: Player Obtained the Key and Reached the Exit Door

9. Display

- Continuously display the number of moves the player has left.
- Show the number of undo moves the player has available at any given time.
- Display the current score, which updates as the player collects coins or completes the game with remaining moves.
- Indicate whether the player has obtained the key. This should be clearly shown once the key has been collected.

- Upon game over, the initial state of the game (with the key and door's locations) must be displayed, along with the coordinates of the collected items.

Figure 7: Game Over

GitHub Repository and Commit Practices

For this assignment, you are required to create a private GitHub repository where you will track your progress and commit code changes regularly. Follow the instructions below to ensure you maintain a professional development workflow.

1. Private Repository

- Create a private repository on GitHub and add your teaching assistants as collaborators to review your progress.
- Ensure that no sensitive information or unnecessary files are included in the repository.

2. Commit Practices

- Commit your progress regularly throughout the development of your game. Each commit should represent a meaningful change, such as the completion of a specific feature or a bug fix.
- Use descriptive commit messages that clearly state the purpose of the commit. For example:
  - `Add movement feature and implement sensing logic`
  - `Fix issue with key sensing distance calculation`
  - `Refactor grid representation to improve performance`

- Avoid committing incomplete or non-functional code unless it is a work-in-progress commit, clearly labeled as such.
- The history of commits will be viewed on the demo day, so ensure consistent progress throughout the assignment.

3. README and Project Documentation

- Create a well-structured `README.md` file in your repository.
- The README should include:
  - A brief introduction to the game.
  - A list of key features and game mechanics.
  - Instructions on how to run the game.
  - Sample outputs or screenshots showing the game in action.
- Properly document any assumptions, libraries, or special configurations needed to run your code.

4. Demo Day

- On the demo day, your commits will be reviewed to ensure consistent progress throughout the development process.
- Be prepared to showcase your repository structure, commit history, and README file as part of the demo presentation.

Submission Criteria and Guidelines

- Submit your code in a single zip file named `DEPARTMENT_SECTION_ROLLNUMBER` (e.g., `DS_A_i211354.zip`). The zip file should contain two files:
  - `DS_A_i211354.cpp` (your code)
  - `i211354_readme` (your README file)
- Use classes for your solution. No marks will be awarded for code that does not use object-oriented programming principles.
- Do not use STL, strings, arrays, or any built-in functions or libraries.
- For Linux/Mac, use the `Ncurses` library. For Windows, use `PDCurses` to display the game.
- If the required output is generated, full marks will be awarded. Failing to produce correct output results in zero marks.
- Zero marks will be given for code that crashes (e.g., segmentation fault) or cannot compile due to syntax errors.

- Plagiarism will result in zero marks for both parties involved. Copying from the internet is strictly prohibited.
- You are not allowed to use LLMs for code generation; however, you can use them for learning. If you do so, you must remember the prompts for the demo day.
- Deadline: October 20, 2024, 11:59 PM. Late submissions will not be accepted.